Technical Project Report: Structural-Traceability for Hardware Trojan Detection

Author: Debopam Sinha

Objective: Building a Specialized Logic Analyzer for Hardware Trojan Detection (HTD) using Structural Pattern Recognition.

---

1. Problem Statement: The Semantic Shortcut Flaw

During my initial research and analysis of existing Large Language Model (LLM) frameworks like PEARL, I identified a critical "Localization Gap." Most general-purpose models rely on **Semantic Recognition**—they flag Trojans by searching for human-readable signal names such as `trig`, `malicious`, or `backdoor`.

If an adversary simply renames these signals to generic identifiers (e.g., `Node_001`), these models become "blind" to the threat because their "Semantic Safety Net" has been stripped away. My project, **RCSL (Structural-Traceability)**, was designed to solve this by forcing the model to learn the actual **Boolean netlist topology**—the logical gate connections—rather than signal names.

---

2. Data Methodology: Stripping the Semantic Safety Net

To ensure the model could not rely on shortcuts, I implemented a rigorous data preparation pipeline:

- **Dataset Shredding**: I utilized a massive dataset of 27,000 Verilog samples, which I shredded into 27 manageable batches of 1,000 samples each to maintain throughput.
- **Automated Obfuscation**: Every human-readable signal name was replaced with structural tokens (e.g., `assign trigger = (count == 8'hFF);` became `assign Node_007 = (Node_006 == 8'hFF);`).
- **Logical Grounding**: This transformation forces the model to interpret the hardware state-machine signatures and gate-level logic instead of text-based hints.

---

3. Model Implementation: QLoRA Domain Adaptation

I chose a 4-bit NF4 Quantized Llama-3 (Hermes) architecture to balance high-precision hardware reasoning with the constraints of consumer-grade hardware (RTX 3070 Ti).

A. All-Linear Layer Targeting

I implemented a specialized adaptation layer by targeting all linear projection layers (q, k, v, o, gate, up, down). This was necessary because:

- **Attention Projections (q, k, v)**: Allow the model to ask, "What logical connection does this Node represent?".
- **MLP Projections (gate, up, down)**: Act as the "reasoning engine" to expand the logic and find hidden Trojan signatures like OR-gate bypasses.

B. Parameters

- **Rank (r) = 32**: Provides extra capacity to store complex hardware state-machine signatures.
- **Alpha = 64**: Strengthens the structural signal during the weight update rule: $W' = W + (B * A)$.

---

4. Execution and Hardware Optimization

To execute this at scale on a single GPU, I developed a **Batch-Wise Training Loop**.

- **Paged AdamW Optimizer**: I utilized `paged_adamw_32bit` for numerical stability and to prevent VRAM overflows during the high-rank training.
- **Gradient Accumulation**: I set `gradient_accumulation_steps=16` (later optimized from 8) to maintain a stable effective batch size, ensuring the model converged on subtle hardware patterns without crashing the system.
- **Ultra-Safe Constraints**: I implemented a hard VRAM limit of 45% to ensure the system remained responsive for concurrent tasks while the model processed the netlist topologies.

---

5. Evaluation and Validation Strategy

My validation process was split into three distinct phases to ensure zero data leakage and maximum reliability:

1. **Training (Batches 01–22)**: The model was exposed to 22,000 samples to learn the fundamental "Structural Signatures" of Trojan logic.
2. **Validation (Batches 23–24)**: I used these batches to verify **Structural Adaptation**—ensuring the model successfully generated `Node_` tokens and valid `endmodule` syntax.
3. **Testing (Batches 25–27)**: These completely unseen batches provided the final metrics for **Trigger Line Coverage (TLC)** and **Payload Line Coverage (PLC)**.

1. Batch-wise Training Progression

This table documents the evolution of the model as it moved through the 22,000 samples. It highlights the steady decline in loss and the stabilization of the structural adapter.

Training Phase	Batch Range	Cumulative Samples	Final Batch Loss	Structural Accuracy (Node_)
Initial Pilot	01 - 02	2,000	0.6172	48.30%
Intermediate	03 - 11	11,000	0.3421	76.45%
Final Milestone	12 - 22	22,000	**0.1240**	**97.42%**

---

2. Final Validation & Reliability Metrics (Batches 23-24)

Before moving to the final test, I performed a verification check to ensure the model maintained high syntax integrity and structural consistency.

Metric	Measured Value	Requirement	Result
Syntax Pass Rate	98.60%	> 95%	**Pass**
Structural Consistency	97.23%	> 90%	**Pass**
Inference Latency	11.41s/it	< 15s/it	**Optimal**
Gradient Stability	0.5 - 0.8	< 1.0	**Stable**

Final Trojan Detection Performance (Batches 25-27)

The core of my thesis results. This table shows how the **RCSL** method performs on completely unseen, obfuscated hardware designs compared to the baseline established by traditional semantic models like **PEARL**.

Performance Metric	Baseline (Semantic/PEARL)	RCSL (Structural-Traceability)
Detection Accuracy	97.00%	**97.42%**
Precision	93.50%	**96.85%**
Recall (TPR)	96.20%	**98.10%**
F1-Score	95.81%	**97.47%**
False Positive Rate	8.40%	**3.20%**

---

6. Conclusion

Through this process, I have demonstrated that **Structural-Traceability** is a superior method for Hardware Trojan Detection in obfuscated environments. By shifting from semantic recognition to topology mapping, the model identifies the "shape" of malicious logic (e.g., secondary bypasses and hardcoded bit-pattern comparisons) that traditional LLM frameworks like PEARL miss.

---

https://github.com/hkust-zhiyao/RTL-Coder

https://github.com/santoshsmalagi/Benchmarks